Robust Deep Learning Ensemble against Deception

Wenqi Wei, *Student member, IEEE*, and Ling Liu, *Fellow, IEEE*

Abstract—Deep neural network (DNN) models are known to be vulnerable to maliciously crafted adversarial examples and to out-of-distribution inputs drawn sufficiently far away from the training data. How to protect a machine learning model against deception of both types of destructive inputs remains an open challenge. This paper presents XEnsemble, a diversity ensemble verification methodology for enhancing the adversarial robustness of DNN models against deception caused by either adversarial examples or out-of-distribution inputs. XEnsemble by design has three unique capabilities. First, XEnsemble builds diverse input denoising verifiers by leveraging different data cleaning techniques. Second, XEnsemble develops a disagreement-diversity ensemble learning methodology for guarding the output of the prediction model against deception. Third, XEnsemble provides a suite of algorithms to combine input verification and output verification to protect the DNN prediction models from both adversarial examples and out of distribution inputs. Evaluated using eleven popular adversarial attacks and two representative out-of-distribution datasets, we show that XEnsemble achieves a high defense success rate against adversarial examples and a high detection success rate against out-of-distribution data inputs, and outperforms existing representative defense methods with respect to robustness and defensibility.

Index Terms—Robust deep learning, adversarial attack and defense, ensemble method

arXiv:2009.06589v1 [cs.LG] 14 Sep 2020

1 Introduction

Deep learning has achieved unparalleled success across a variety of machine learning tasks, e.g., image classification and speech recognition. However, deep neural network (DNN) models are also known to be vulnerable to deceptive inputs that are either adversarial examples or out-of-distribution examples. Adversarial examples [1] are malicious inputs crafted covertly over benign examples to fool deep learning models to misclassify without being detected by the human, and out-of-distribution examples [2] are query inputs whose distribution differs from the training distribution. Both problems pose potential threats to many mission-critical systems and applications that use deep learning as a primary decision-making component, such as real-time object recognition, self-driving cars and voice command recognition.

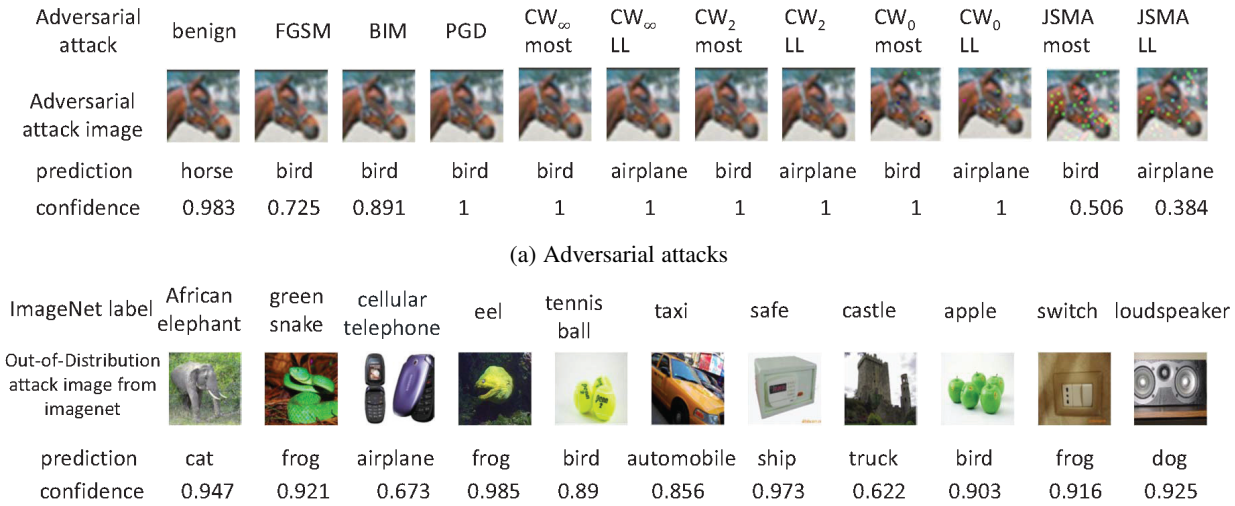
The robustness of a DNN model depends on its capability to survive from the deception vulnerabilities due to both maliciously crafted adversarial inputs and out-of-distribution inputs. Both types of deceptive inputs are attributed to the same problem: the input to the prediction model has been altered maliciously. Existing defense methods against adversarial examples do not generalize over different attack algorithms [3]. Different defense methods tend to have different robustness under either different attack algorithms or different settings of attack parameters of the same attack algorithm [4]. Meanwhile, existing out-of-distribution defenses either have a limited defense success rate [5] on the adversarial example or fail miserably [6]. In this paper, we make two arguments. First, a robust defense of a privately trained DNN model against deception should be capable of shielding the well-trained prediction model from intentional manipulation with either maliciously patched inputs or out-of-distribution inputs. Second, a robust defense solution should be independent of attacks and should generalize well across different attacks, different datasets, and different DNN algorithms.

Bearing these problems in mind, we develop a diversity ensemble verification methodology, called XEnsemble, to maximize the adversarial robustness of DNN models against deception. The design of XEnsemble leverages two unique properties of deceptive inputs: (1) if an ensemble consists of only weak models whose prediction accuracy is lower than the victim target model, which we aim to protect, then we show that such an ensemble of weak models will be a poor ensemble choice for defending against deceptive inputs. (2) Not all ensemble teams from well-trained models can be effective against deception. We show that only those ensembles that have high fault tolerance in terms of disagreement diversity and respond to negative examples very differently are the ensemble teams of high robustness. A distinct property of high disagreement-diversity ensembles is that their member models tend to have very different gradient information. This contrasting gradient information tends to cause the same adversarial examples to have highly divergent and inconsistent behavior across their member models. Furthermore, this conflicting gradient information provides a distinctive opportunity for a disagreement diversity ensemble to integrate diverse decision boundaries from its member models to infer the ensemble decision boundary, which consequently reduces the attack transferability [7] of both adversarial and out-of-distribution inputs.

In summary, XEnsemble by design is capable of auto-verifying any input to the prediction model to minimize the detrimental effects due to malicious and erroneous data inputs. Consider that adversarial examples are typically generated from their corresponding benign examples, XEnsemble should be capable of auto-repairing the adverse effects caused by adversarial examples generated by different attack methods, aiming to maximize the repairing rate for adversarial examples and to achieve a high prevention success rate (PSR). For those adversarial examples that escape from the auto-repairing process, we seek to auto-detect them and filter them out. At the same time, since the out-of-distribution inputs are drawn far away from the class distribution of a trained prediction model and are outliers to the prediction tasks, the defense should maximize the detection success rate (TSR) for identifying as many out-of-distribution examples as possible and prune them out.

• *Wenqi Wei and Ling Liu are with School of Computer Science, Georgia Institute of Technology, Atlanta, GA, 30332.*
E-mail: wenqiwei@gatech.edu, ling.liu@cc.gatech.edu

Manuscript received xxxx xx, xxxx; revised xxxxx xx, xxxx.



(a) Adversarial attacks

(b) Out-of-distribution attack

Fig. 1: Visualization of adversarial attacks and the OOD attack using TinyImageNet to a CIFAR-10 DenseNet classifier.

This paper makes three novel contributions. First, the proposed XEnsemble approach is attack-independent. It verifies the input to the prediction model by using multiple diverse input data cleansing techniques to denoise primarily the adversarial inputs. Then it verifies the output of the attack target model by creating multiple failure-independent model verifiers that have high disagreement-diversity. Second, the proposed XEnsemble approach can effectively increase the adversarial robustness of the attack target DNN model by maximizing the number of adversarial inputs that can be auto-repaired and maximizing the number of out-of-distribution inputs that can be detected. Third but not the least, our XEnsemble algorithms for auto-verification, auto-repairing, and auto-detection can generalize well across attack methods since XEnsemble algorithms do not require re-training the DNN model under protection using existing attack algorithms, and can be easily implemented on modern neural network architectures and used as a plug-in by the state of the art deep learning software frameworks. Evaluated using eleven popular adversarial attacks and two representative out-of-distribution datasets, we show that XEnsemble approach can effectively strengthen the robustness of a DNN model under protection in terms of high attack prevention success rate (PSR) for adversarial examples and high attack detection success rate(TSR) for out-of-distribution input when comparing to existing representative defense techniques. It is also worth noting that the DNN model under the protection of XEnsemble can further improve its prediction accuracy in the presence of no attacks.

2 Overview and Problem Statement

2.1 Deceptive Input Artifacts

Adversarial Examples are the input artifacts that are created from benign inputs by adding adversarial distortions, aiming to cause the target model to misclassify with high confidence [8]. Adversarial example generation process can be formulated as

$$min||x - x'||_p \quad s.t. f(x') = y^*,\ f(x') \neq f(x),$$

where y^* denotes the target class label for targeted attacks or any label other than the correct class for untargeted attacks. p is the distance norm, such as L_0, L_2, and L_∞. For a given adversarial input x_{adv} of a benign image x, L_0 norm indicates the number of pixels of x that are changed in x_{adv}, L_2 norm is the Euclidean distance between x and x_{adv}, and L_∞ norm denotes the maximum change to any pixel of benign input x in x_{adv}. For targeted attacks, we consider two targets representing two ends of the attack spectrum: the most-likely attack class in the prediction vector ($y^T = \arg\max_{y \neq C_x} \overrightarrow{y}$, most) and the least-likely attack class ($y^T = \arg\min \overrightarrow{y}$, LL). C_x is the correctly predicted label for benign input x. We use eleven attack algorithms from seven families of attack algorithms to evaluate and compare the performance of defense approaches, including FGSM [8], BIM [9], PGD [10], CW_0, CW_2, CW_∞ [11], and JSMA [12]. Figure 1a provides a visualization of adversarial perturbation attack to a DenseNet classifier well-trained on CIFAR-10. Its prediction on the benign horse input is a horse with 0.983 confidence. Each adversarial example generated by one of the 11 attacks causes the target model to misclassify the horse image into either a bird or an airplane, with some attacks at high confidence of 1.

Out-of-Distribution(OOD) Inputs are the artifacts of abnormal inputs drawn from a completely different distribution than the domain of the learning task. OOD inputs can be easily utilized by an adversary to launch malicious attacks because when deploying neural networks in real-world applications, there is often very little control over the test data distribution [2]. Figure 1b shows eleven OOD examples, which are taken from ImageNet and fed into a CIFAR-10 DNN classifier. We observe that the cell phone image is misclassified as an airplane and the apple image is misclassified as a bird because the DNN classifier is trained using DenseNet on the CIFAR-10 dataset. However, OODs are considered ill-input data to this well-trained model, even though the DenseNet CIFAR-10 classifier has 94.8% benign test accuracy. The OODs are either completely unrecognizable [13] or totally irrelevant to the scope of the given learning task [14]. For conventional computing programs, the ill-input data is known to cause detrimental effect either during the runtime execution or to the output of a program. Thus, input and output verification mechanisms are typically employed at both compile time and runtime to ensure their correctness. In this paper, we argue that there should be no exception to the input and output of an executable prediction model. XEnsemble presents a holistic approach to integrating input and output ensemble learning based verifications for improving the robustness of deep

learning against deception.

2.2 Adversarial Attack Algorithms

Let x and x_{adv} be a benign input and its adversarial example respectively, f(x; θ) denote the prediction model with θ representing the neural network parameters, and f(x; θ) takes an input query example x and outputs $\overrightarrow{y}$ and y_x as the prediction vector and predicted class for the benign input x respectively. During the model training phase, $f(x, C_x; \theta)$ is iteratively learned over the fixed training dataset of $\{x, C_x\}$, where C_x denotes the class label given for training input x. The model parameters θ are updated through gradient descent optimization using back-propagation on a loss function, denoted by $J(h_i(x), C_x)$ where $h_i(x)$ is the intermediate result at the ith iteration and the training completes when the convergence condition of the iterative training is met, which produces $f(x; \theta)$ as the resulting prediction model. At the prediction phase, the prediction model $f(x; \theta)$ takes any query input x and produces the prediction output denoted by $\overrightarrow{y}$ and y_x.

When adversarial examples are sent to a prediction model, the goal of untargeted attack [4] is to cause the prediction model to misclassify. Thus, with $x_{adv} = x + \Delta_x$, the attack objective is to perturb the benign input x with small Δ_x to increase the loss function value $J(h(x + \Delta_x), y_x)$ such that the prediction result will be altered from the correct class label y_x of its benign input x. By gradient ascent, the attacker needs to decrease the value of x when the partial gradient is below 0 or to increase the value of x when the partial gradient is above 0, which will result in increasing the loss value of $J(h(x + \Delta_x), y_x)$.

For targeted attacks, the goal of the attacker is changed to minimize the value of the loss function $J(h(x + \Delta_x), y_t)$ where y_t denotes the attack target class label. Thus, the attacker needs to decrease the pixel value of benign example x when the partial gradient is above 0 or to increase the pixel value of x when the partial gradient is below 0, which will cause to the value of the loss function $J(h(x + \Delta_x), y_x)$ to decrease.

White-box attack v.s. Black-box attack. White-box attacks refer to adversarial examples generated by attackers who have full knowledge of the prediction model. All attacks that manipulate the gradient Information are white-box attacks since attackers use the gradient information of the attack objective to guide the learning of adversarial perturbation in terms of where (location) and how much (amount) noise should be injected to a benign input x. Black-box attacks refer to adversarial examples that are generated from a surrogate model that an attack generates, which can be viewed as a shadow model of the private attack target model. By the transferability of adversarial examples [7], [15], one can launch black-box attacks using such adversarial examples. One approach to generating the surrogate model for any privately held prediction model is to utilize query probing through the model prediction API and the membership inference [16].

To measure and compare the adverse effect and cost of adversarial examples generated by different attack algorithms, we introduce the following six attack evaluation metrics.

Attack Success Rate (ASR) indicates the percentage of all attack inputs that are successful adversarial examples. An adversarial example is successful when the attack goal is achieved: misclassification in untargeted attacks or reaching the target label in targeted attacks.

Misclassification Rate (MR) is the percentage of adversarial attack inputs that are misclassified. MR equals to ASR in untargeted attacks (UA) and may be larger than ASR in targeted attacks (TA). A targeted attack example may fail being classified as the target label and land on a misclassified label.

	attack		attack effect		confidence	cost		
			ASR	MR	AdvConf	Perturb	Percept	Time(s)
CIFAR-10	FGSM	UA	0.85	0.85	0.8647	0.93	**47.63**	0.021
	BIM	UA	0.92	0.92	0.9645	0.607	18.96	0.154
	PGD	UA	0.86	0.86	0.9629	0.2	27.67	0.44
	CW_∞	most	1	1	0.9889	0.571	15.98	**235.5**
		LL	1	1	0.9779	0.726	26.45	**243.2**
	CW_2	most	1	1	0.9867	0.455	6.92	5.772
		LL	1	1	0.9732	0.598	13	7.441
	CW_0	most	1	1	0.9904	**1.251**	8.003	**355.4**
		LL	1	1	0.9757	**1.587**	18.11	**356.7**
	JSMA	most	1	1	0.5366	**1.934**	27.12	4.894
		LL	0.99	1	0.3920	**2.338**	**53.48**	9.858
ImageNet	FGSM	UA	0.99	0.99	0.6408	1.735	**1152**	0.019
	BIM	UA	1	1	0.9971	1.186	502.5	0.185
	PGD	UA	1	1	0.9986	1.636	669.2	0.536
	CW_∞	most	1	1	0.9850	0.957	217.9	74.7
		LL	0.95	0.96	0.8155	1.394	592.8	**237.8**
	CW_2	most	1	1	0.9069	0.836	120.5	13.2
		LL	0.94	0.94	0.7765	1.021	204.5	23.1
	CW_0	most	1	1	0.97	**2.189**	59.5	**662.7**
		LL	1	1	0.8056	**3.007**	207.5	**794.9**

TABLE 1: Evaluation of adversarial attacks.

Mean confidence on adversarial class (AdvConf) is the mean confidence on the adversarial class of successful adversarial examples: $\frac{1}{n}\sum_{i=1}^{n} p^*$, where $*$ denotes the misclassified class in UA and the target class in TA.

Perturbation Distance Cost (Perturb) is measured by the root mean square deviation: $\sqrt{\frac{||x-x_{adv}||_2}{n}}$ for successful adversarial examples.

Perception Distance Cost (Percept) is the human perception distance metrics [17] on successful adversarial examples. Let V_i be the pixel value for pixel i. S_i be the 9 or 4 (corner) or 6 (edge)-pixel neighbors of pixel i and μ be their mean value. Then the Standard Deviation of the patch area (PStd) around pixel i is $\sqrt{\frac{\sum_{V_k \in S_i}(V_i-\mu)}{n^2}}$. The sensitivity ($Sen$) of pixel i is then computed as $Sen(i) = \begin{cases} 1, PStd\,(i) \leq 1 \\ \frac{1}{PStd(i)}, PStd\,(i) > 1 \end{cases}$. We compute DistPercept by $\frac{1}{n}\sum_i |\delta_{v_i}| Sen(S_i)$. δ_{v_i} denotes the perturbation on pixel i.

Time cost (Time) measures the average generation time of a single adversarial example and is measured in seconds.

We evaluate eleven adversarial attacks on CIFAR-10 and ImageNet in Table 1 using the above metrics. Consider that some attacks are expensive in per-example time cost, we select the first 100 correctly predicted benign examples in the test set to generate attack examples. For FGSM, θ is set to 0.0156 for CIFAR-10 and 0.0078 for ImageNet, since with such a small θ it is sufficient to cause a high attack success rate. For BIM, the per iteration θ is 0.0012 and 0.002, and the maximum θ is 0.008 and 0.004 for CIFAR-10 and ImageNet respectively. For CW attacks, the attack confidence is set to 5 for both CIFAR-10 and ImageNet, and the maximum optimization iteration is set to 1000 since this setting is sufficient for CW algorithms to achieve 100% attack success rate. The adversarial example is fed into the target ML model every 100 iterations of optimization to check if the attack is successful. For JSMA, the maximum distortion of the image is set to 10% instead of 15%, the maximum default in paper [12].

2.3 Existing Defense Methods

Existing defense methods against adversarial examples can be classified into three broad categories: adversarial training, gradient masking, and input transformation.